

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3 namespace cs225 {
4     class Sphere {
5     public:
6         Sphere(); default constructor
7         Sphere(double r);
8         double getRadius();
9         double getVolume();
10
11
12
13
14     private:
15         double r_;
16
17     };
18 }
19
20 #endif
```

sphere.cpp

```
1 #include "sphere.h"
2 namespace cs225 {
3     Sphere::Sphere() call this custom constructor : Sphere(1.0) {
4     }
5
6
7     Sphere::Sphere(double r) {
8         r_ = r;
9         // ...lots of other logic...
10    }
11
12    double Sphere::getRadius() {
13        return r_;
14    }
15
16    double Sphere::getVolume() {
17        return (4 * r_ * r_ * r_ *
18                3.14159265) / 3.0;
19    }
20 }
```

sphere.h

```

1  #ifndef SPHERE_H
2  #define SPHERE_H
3  namespace cs225 {
4      class Sphere {
5      public:
6          Sphere(double r=1.0);
7          double getRadius();
8          double getVolume();
9
10
11
12
13     private:
14         double r_;
15
16     };
17 }
18
19 #endif
20

```

just like in python

eg. Sphere S2;
 it will call this

Sphere S3(>);
 actually declare a function

sphere.cpp

```

1  #include "sphere.h"
2  namespace cs225 {
3      simply use one constructor
4      Sphere::Sphere(double r) {
5          r_ = r;
6          // ...lots of other logic...
7      }
8
9      double Sphere::getRadius() {
10         return r_;
11     }
12
13     double Sphere::getVolume() {
14         return (4 * r_ * r_ * r_ *
15                3.14159265) / 3.0;
16     }
17 }
18
19
20

```

sphere.h

```
1 #ifndef SPHERE_H
2 #define SPHERE_H
3 namespace cs225 {
4     class Sphere {
5     public:
6         Sphere(double r=1.0);
7         double getRadius();
8         double getVolume();
9
10
11
12
13     private:
14         double r_;
15
16     };
17 }
18
19 #endif
20
```

sphere.cpp

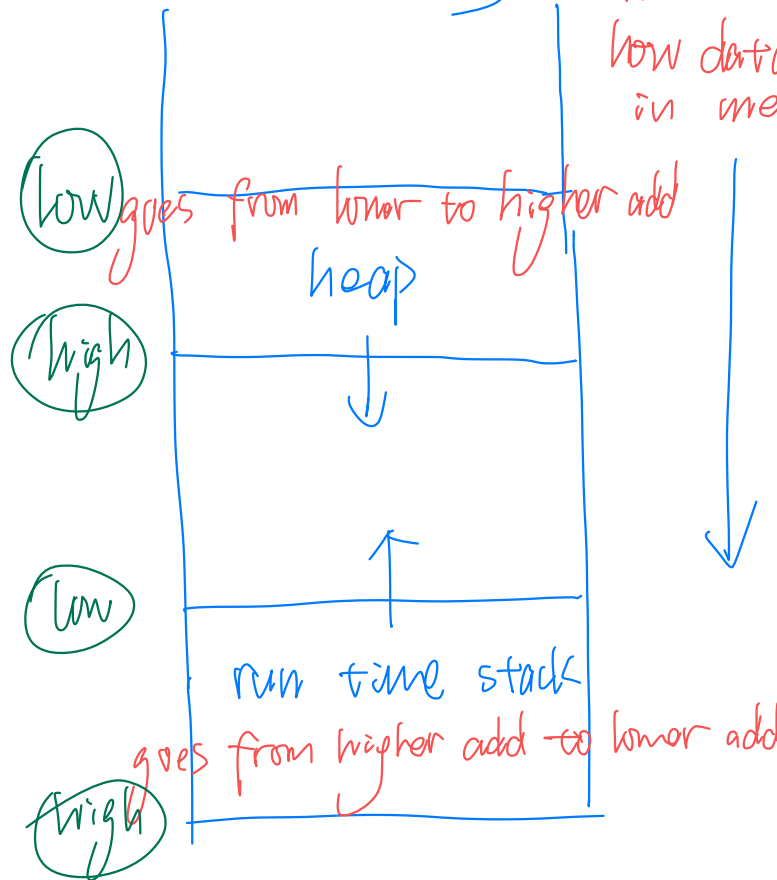
```
1 #include "sphere.h"
2 namespace cs225 { 初始化成员列表
3     initialization list
4     Sphere::Sphere(double r):r_(r)
5     {
6         // ...lots of other logic...
7     }
8
9     double Sphere::getRadius() {
10         return r_;
11     }
12
13     double Sphere::getVolume() {
14         return (4 * r_ * r_ * r_ *
15                 3.14159265) / 3.0;
16     }
17 }
18
19
20
```

Memory "Types"

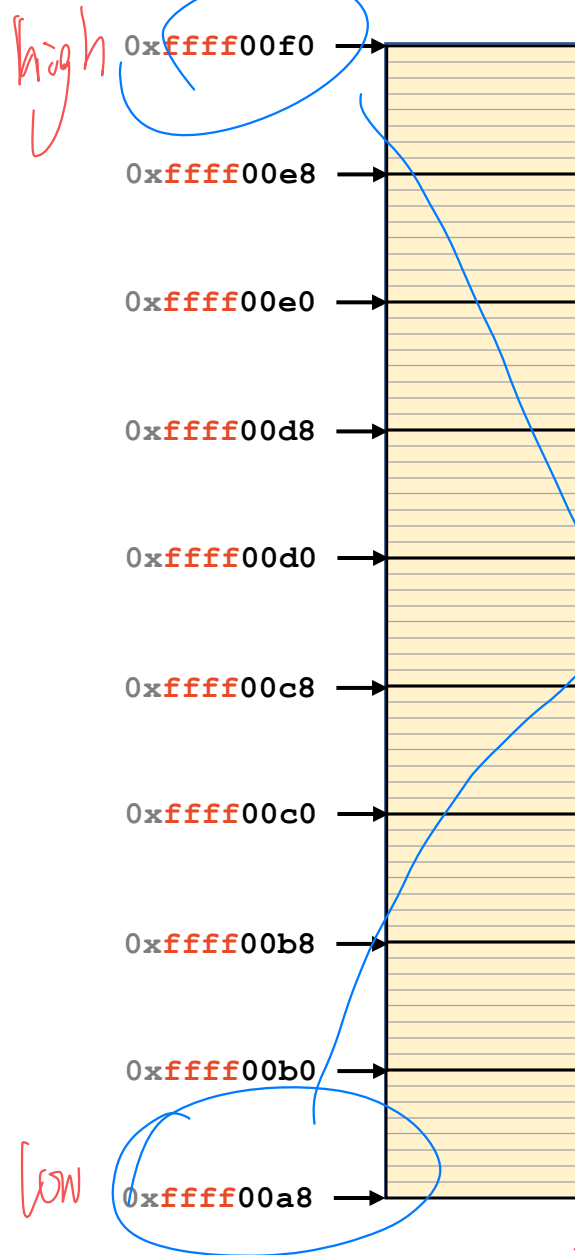
how app store objects in memory

memory

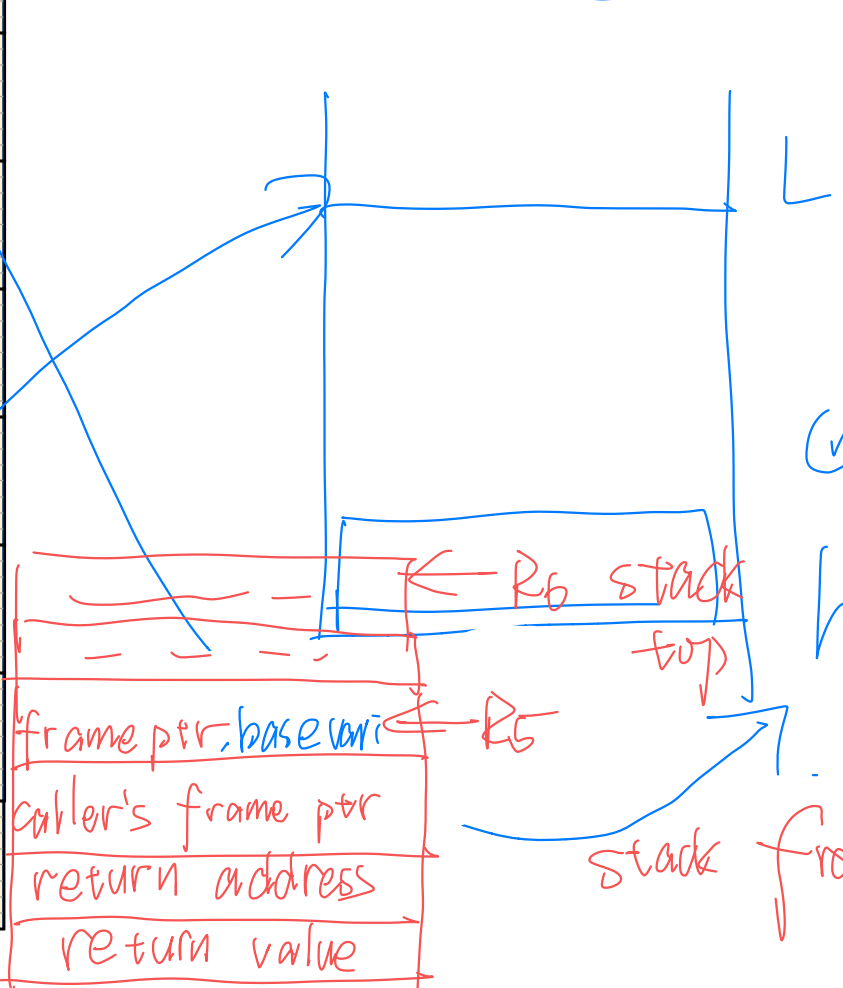
now, look in detail
how data is organized
in memory



Stack Memory



when declare variable in cpp function,
it's placed in run time stack



1) variables are located on the stack

2) stack space are organized from high add to low add

stack frame pointer: ptr to the base / first variable

example1.cpp

```

1 int main() {
2     int a;
3     int b = -3;
4     int c = 12345;
5
6     int *p = &b;
7
8     return 0;
9 }

```

	Location	Value	Type	Name
high	0xffff00f0	a		
	0xffff00e8	b = -3		
	0xffff00e0	c = 12345		
	0xffff00e0	p = 0xffff00e8		
	0xffff00d8			
	0xffff00d0			
	0xffff00c8			
	0xffff00c0			
	0xffff00b8			
	0xffff00b0			
low	0xffff00a8			

sizeof-int.cpp

```
1 #include <iostream>
2
3 int main() {
4     std::cout << sizeof(int) << std::endl;
5     return 0;
6 }
```


sizeof-intptr.cpp

```
1 #include <iostream>
2
3 int main() {
4     std::cout << sizeof(int *) << std::endl;
5     return 0;
6 }
```

example1.cpp

```

1 int main() {
2     int a;
3     int b = -3;
4     int c = 12345;
5
6     int *p = &b;
7
8     return 0;
9 }

```

<u>Location</u>	<u>Value</u>	<u>Type</u>	<u>Name</u>
0x7ffe2ee87228	→		
0x7ffe2ee87220	→		
0x7ffe2ee87218	→		
0x7ffe2ee87210	→		
0x7ffe2ee87208	→		
0x7ffe2ee87200	→		
0x7ffe2ee871f8	→		
0x7ffe2ee871f0	→		
0x7ffe2ee871e8	→		
0x7ffe2ee871e0	→		



Real results when running on a 64-bit system

&a: 0x7ffe2ee87218
 &b: 0x7ffe2ee87214
 &c: 0x7ffe2ee87210
 &p: 0x7ffe2ee87208

4 bytes

4 bytes

4 bytes

high address
 ↓
 stack
 ↓
 low address

example2.cpp

<u>Location</u>	<u>Value</u>	<u>Type</u>	<u>Name</u>
0xffff00f0 →			
0xffff00e8 →			
0xffff00e0 →			
0xffff00d8 →			
0xffff00d0 →			
0xffff00c8 →			
0xffff00c0 →			
0xffff00b8 →			
0xffff00b0 →			
0xffff00a8 →			

```

1 #include "sphere.h"
2
3 int main() {
4     cs225::Sphere s;
5     cs225::Sphere *p = &s;
6
7     return 0;
8 }
9

```

Sphere : only one attributes

sizeof-sphere.cpp

```
1 #include <iostream>
2 #include "sphere.h"
3
4 int main() {
5     std::cout << sizeof(cs225::Sphere) << std::endl;
6     std::cout << sizeof(cs225::Sphere*) << std::endl;
7     return 0;
8 }
```

8 bytes
8 bytes

*int * is also 8 bytes*

Stack Frames



with gdb to
see what
happened in run
time stack

```

1  int hello() {
2      int a = 100;
3      return a;
4  }
5
6  int main() {
7      int a;
8      int b = -3;
9      int c = hello();
10     int d = 42;
11
12     return 0;
13 }

```

- 1) stack frame is created when function is called
- 2) stack frame is destroyed when exiting the function

Return Values

by default, in C++ - return values are copied

sphere-badCreate.cpp

Location	Value	Type	Name
0xffff00f0			
0xffff00e8	ptr		
0xffff00e0	S = 0xffff00db		
0xffff00d8	sphere		
0xffff00d8			
0xffff00d0			
0xffff00c8			
0xffff00c0			
0xffff00b8			
0xffff00b0			
0xffff00a8			

```

1 #include "sphere.h"
2 using namespace cs225;
3
4 Sphere *CreateUnitSphere() {
5     Sphere s(1);
6     return &s;
7 }
8
9 int main() {
10     Sphere *s = CreateUnitSphere();
11     double r = s->getRadius();
12     double v = s->getVolume();
13     return 0;
14 }

```

by chance,
variable s
has been
destroyed in

run time stack, and it's value may be modified

在这里, s 这个 sphere 的 object 刚创建还没有被改,
r 也是一个 double, 在 CreateUnitSphere 创建的时候
in object 的位置?



What happens on a real system?


```

13 int main() {
14     Sphere *s = CreateUnitSphere();
15     cout << s->getRadius() << endl;
16     cout << "s->getRadius(): "
17           << s->getRadius() << endl;
18     cout << "&s (main): " << &s << endl;
19     cout << " s (main): " << s << endl;
20     double r = s->getRadius();
21     cout << "&r (main): " << &r << endl;
22     cout << " r (main): " << r << endl;
23     double v = s->getVolume();
24     cout << "&v (main): " << &v << endl;
25     cout << " v (main): " << v << endl;
26 }

```

Real results when running on linus.ews.illinois.edu

```

&s (CreateUnitSphere): 0x7ffee6bf5ca8
1
s->getRadius(): 2.07941e-317

```

```

&s (main): 0x7ffee6bf5d30
s (main): 0x7ffee6bf5ca8

```

```

&r (main): 0x7ffee6bf5d28
r (main): 6.95312e-310

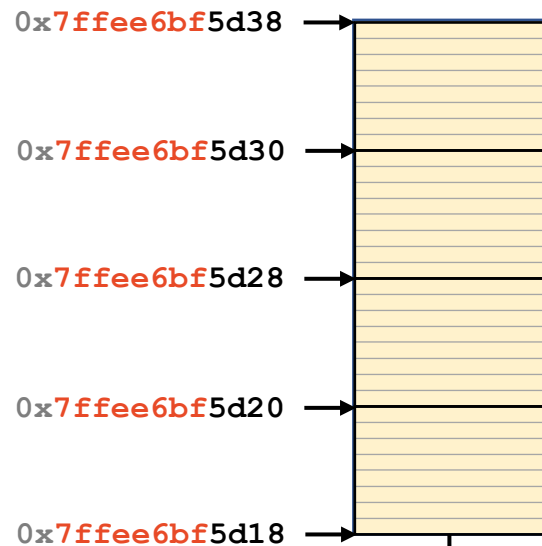
```

```

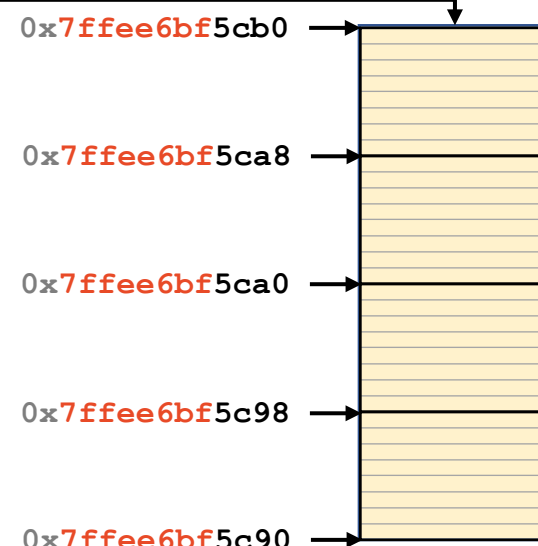
&v (main): 0x7ffee6bf5d20
v (main): 033

```

in some machine



(0x60 bytes not shown)



*in other machine,
it doesn't behave
exactly like this,
even the stack
behavior*

Stack Memory



vs.

Heap Memory

we need to
create it on heap

grow to
higher
address



c++ operator

'new'

doesn't equal to
malloc,

malloc: 分配内存

new: 分配, 还 call
constructor

在c++里不使用
malloc

delete , not free

stack-array.cpp

<u>Location</u>	<u>Value</u>	<u>Type</u>	<u>Name</u>
0xffff00f0 →			
0xffff00e8 →			
0xffff00e0 →			
0xffff00d8 →			
0xffff00d0 →			
0xffff00c8 →			
0xffff00c0 →			
0xffff00b8 →			
0xffff00b0 →			
0xffff00a8 →			

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int first = 42;
6      int arr[6];
7
8      cout << &first << endl;
9      cout << &(arr[0]) << endl;
10     cout << &(arr[1]) << endl;
11     cout << &(arr[2]) << endl;
12
13     return 0;
14 }
```

stack-array.cpp

Location	Value	Type	Name
0xffff00f0			
0xffff00e8			
0xffff00e0			
0xffff00d8			
0xffff00d0			
0xffff00c8			
0xffff00c0			
0xffff00b8			
0xffff00b0			
0xffff00a8			

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int first = 42;
6      int arr[6];
7
8      cout << &first << endl;
9      cout << &(arr[0]) << endl;
10     cout << &(arr[1]) << endl;
11     cout << &(arr[2]) << endl;
12
13     return 0;
14 }

```

```

waf@linux-a2:~
[waf@linux-a2 ~]$ clang++ -std=c++11 -stdlib=libc++ stack-array.cpp
[waf@linux-a2 ~]$ ./a.out
0x7ffc8c8ef4e8
0x7ffc8c8ef4d0
0x7ffc8c8ef4d4
0x7ffc8c8ef4d8
[waf@linux-a2 ~]$ nano stack-array.cpp
[waf@linux-a2 ~]$

```

↑ 地址地址
↓ 上到下，跟市井的例子

stack-array.cpp

Location	Value	Type	Name
0x7ffc8c8ef4f0			
0x7ffc8c8ef4e8	42	int	first
0x7ffc8c8ef4e0	arr[5]		
0x7ffc8c8ef4d8	arr[4]		
0x7ffc8c8ef4d8	arr[3]		
0x7ffc8c8ef4d8	arr[2]	0x7ffc8c8ef4d8	
0x7ffc8c8ef4d8	arr[1]	0x7ffc8c8ef4d4	
0x7ffc8c8ef4d0	arr[0]	0x7ffc8c8ef4d0	
0x7ffc8c8ef4c8			
0x7ffc8c8ef4c0			
0x7ffc8c8ef4b8			
0x7ffc8c8ef4b0			
0x7ffc8c8ef4a8			

真实的 behavior
不是这样, 完全
是 compiler
自己决定的

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int first = 42;
6     int arr[6];
7
8     cout << &first << endl;
9     cout << &(arr[0]) << endl;
10    cout << &(arr[1]) << endl;
11    cout << &(arr[2]) << endl;
12
13    return 0;
14 }

```

```

waf@linux-a2:~
[waf@linux-a2 ~]$ clang++ -std=c++11 -stdlib=libc++ stack-array.cpp
[waf@linux-a2 ~]$ ./a.out
0x7ffc8c8ef4e8
0x7ffc8c8ef4d0
0x7ffc8c8ef4d4
0x7ffc8c8ef4d8
[waf@linux-a2 ~]$ nano stack-array.cpp
[waf@linux-a2 ~]$

```

```

1 #include "sphere.h"      heap1.cpp
2 using namespace cs225;
3
4 int main() {
5     int *p = new int;
6     Sphere *s = new Sphere(10);
7
8
9
10    return 0;
11 }

```

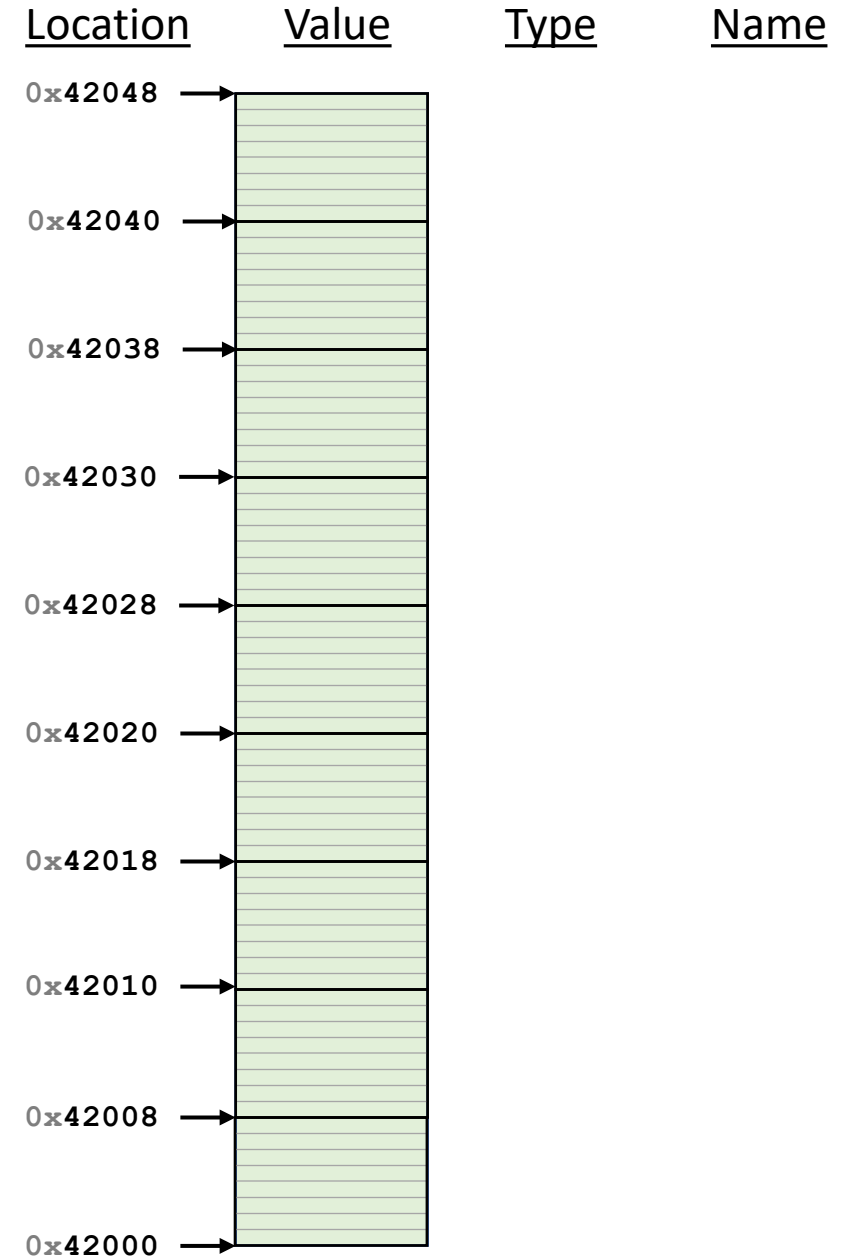
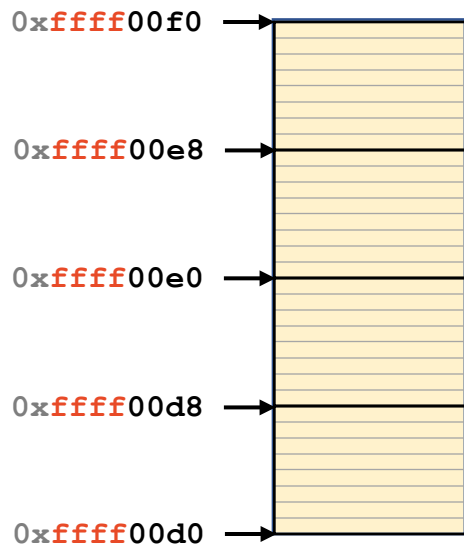
<u>Location</u>	<u>Value</u>	<u>Type</u>	<u>Name</u>
0xffff00f0 →			
0xffff00e8 →			
0xffff00e0 →			
0xffff00d8 →			
0xffff00d0 →			

<u>Location</u>	<u>Value</u>	<u>Type</u>	<u>Name</u>
0x42048 →			
0x42040 →			
0x42038 →			
0x42030 →			
0x42028 →			
0x42020 →			
0x42018 →			
0x42010 →			
0x42008 →			
0x42000 →			

```

1  #include "sphere.h"      heap2.cpp
2  using namespace cs225;
3
4  int main() {
5      Sphere *s1 = new Sphere();
6      Sphere *s2 = s1;
7
8      s2->setRadius( 10 );
9
10
11
12     return 0;
13 }

```



Heap Memory Lifecycle

① create new operator $\in$ P } C++

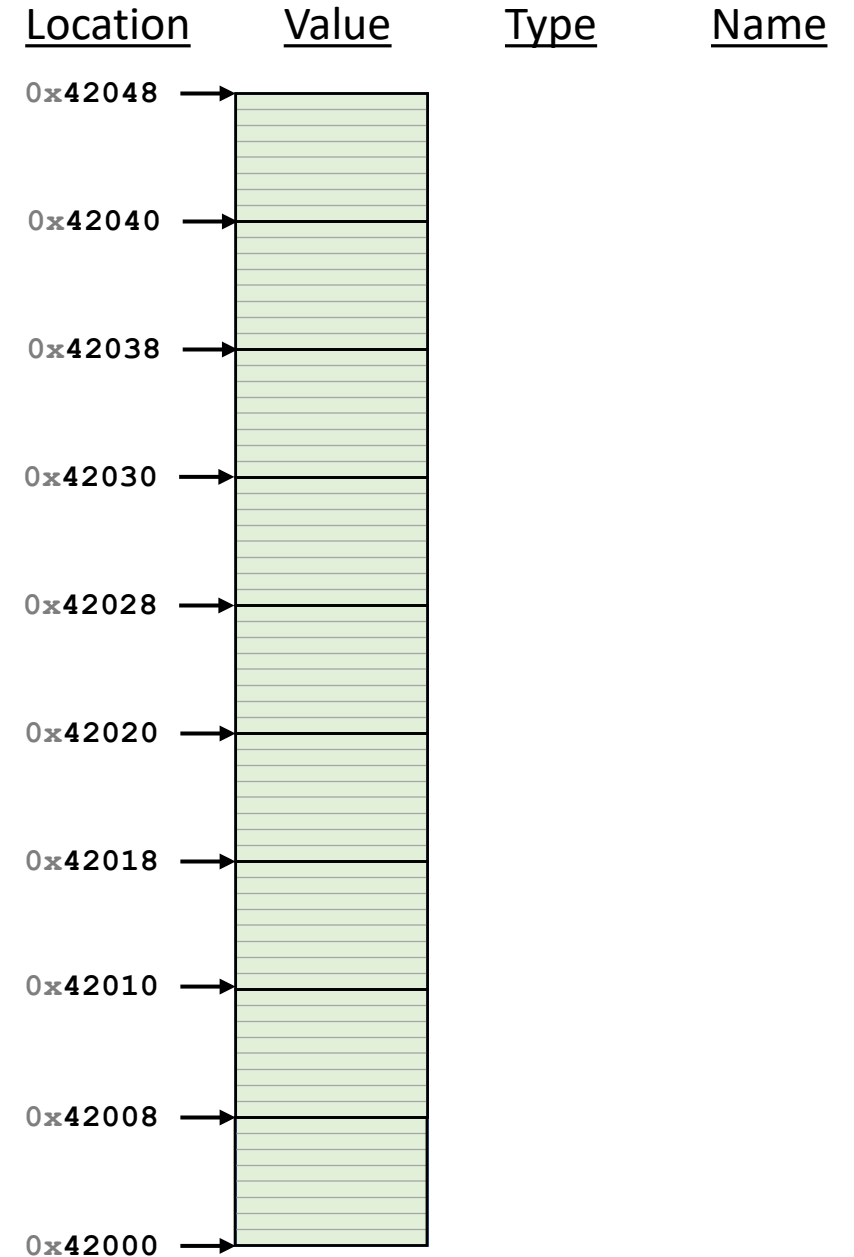
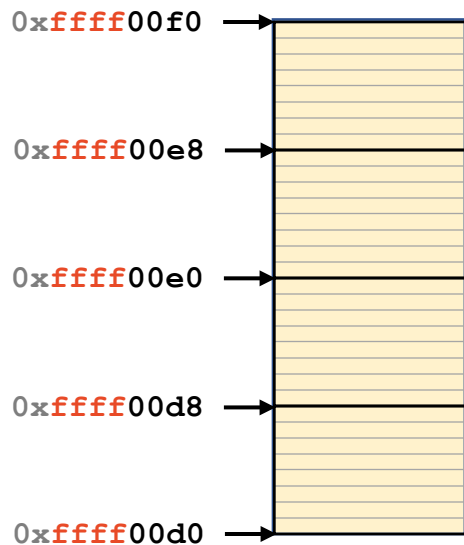
② reclaim: delete $\leftarrow$ P $\leftarrow$ null ptr (NULL in C)

After deletion,
still have a pointer, still has access to
this location (segfault)


```

1  #include "sphere.h"      heap2.cpp
2  using namespace cs225;
3
4  int main() {
5      Sphere *s1 = new Sphere();
6      Sphere *s2 = s1;
7
8      s2->setRadius( 10 );
9
10     delete s1;
11     s1 = nullptr; s2 = nullptr;
12     return 0;
13 }

```



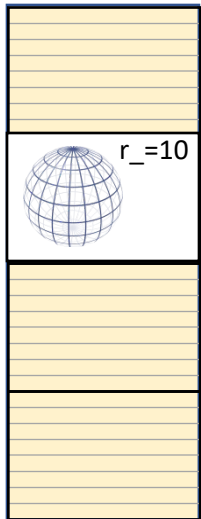
Accessing Memory and Data

&S

address operator

get the address

operator

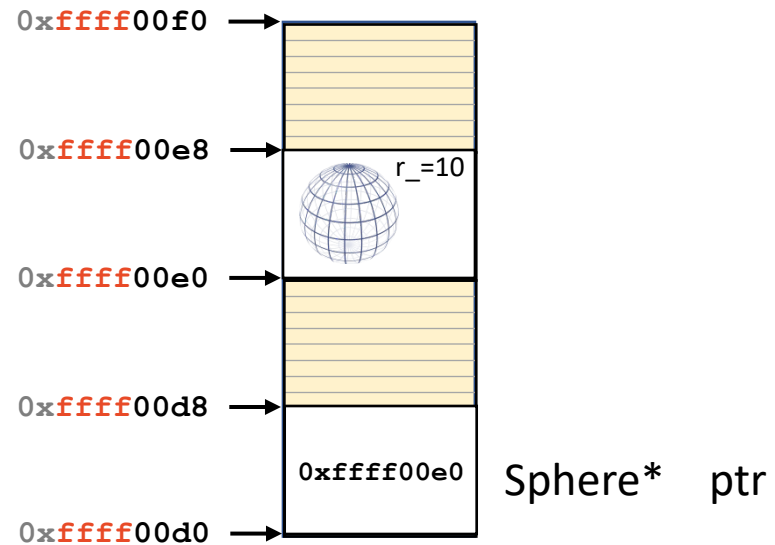


Sphere s

Accessing Memory and Data

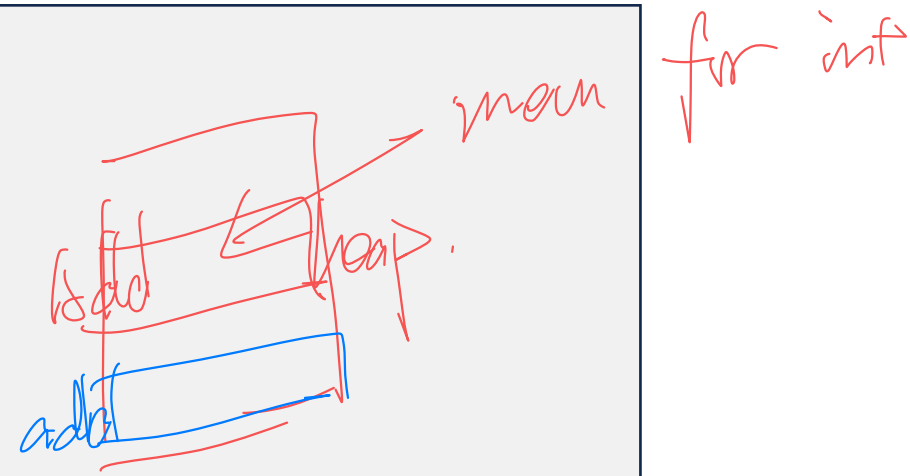
***ptr**

in direct ion operation



heap-puzzle1.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int *p, *q;
6     p = new int;
7     q = p;
8     *q = 8;
9     cout << *p << endl;
10
11     q = new int;
12     *q = 9;
13     cout << *p << endl;
14     cout << *q << endl;
15
16     return 0;
17 }
```



p, q = add
**[add] = 8*

q = add
**[add] = 9*

heap-puzzle2.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int *x;
6      int size = 3;
7
8      x = new int[size];
9
10     for (int i = 0; i < size; i++) {
11         x[i] = i + 3;
12     }
13
14     delete[] x;
15 }
16
17
```

CS 225 – Things To Be Doing

lab_intro Due next Sunday by 11:59pm

Make sure to turn in lab_intro by Sunday at 11:59pm

MP1 is out soon *(after labs have finished)*

Due: Monday, March 2nd (when you return from the winter break)

after 11:59?

Post questions on Piazza ...

- *Having trouble with C++?*
- *Having trouble with lab_intro?*
- *Don't understand something from lecture?*

Have an awesome weekend!

...see you next week!